Git and version control

Track changes with Git: commits, branches, merging, rebase, remotes, pull requests, tags, hooks, large repositories, and recovery when things go wrong.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Your first repository: snapshots, commits and history (12 min)
2. Undoing changes safely: restore, amend, revert, reset and stash (13 min)
3. Branches and merging (13 min)
4. Rebase and a clean history (13 min)
5. Remotes: clone, fetch, pull and push (13 min)
6. Team workflows: branching strategies, pull requests and reviews (14 min)
7. Tags, releases and version numbers (12 min)
8. Hooks, signed commits and secrets in history (14 min)
9. Large repositories, submodules and worktrees (13 min)
10. Debugging Git problems: reflog, bisect, blame and recovery (14 min)
11. Git internals: objects, refs and the index (12 min)
12. Release and hotfix workflows with Git (12 min)
Cheat sheet

Lesson 1. Your first repository: snapshots, commits and history

12 minutes. Understand what version control records, make your first commits with a clear message, read the history, and keep generated files out with .gitignore.

You have a folder of scripts and a file called deploy_final_v2_REAL.sh next to deploy_final_v3.sh. Nobody knows which one runs in production or what changed between them. Version control replaces that folder-copy habit with a complete, searchable history of every change, who made it and why. Git is the tool almost every team uses for it, and its core idea is small enough to learn in one sitting.

What you will learn

- What Git stores: snapshots, commits and a history you can search
- The cycle: edit, stage with git add, save with git commit
- Read the history with git log, git diff and git show
- Keep build output and secrets out of the repository with .gitignore

Snapshots, not copies of files

Git is a **distributed** version control system. Every copy of a repository (a **clone**) holds the full history, so you can commit, compare and browse the past without a network. Each **commit** is a snapshot of the whole project at one moment, plus a message, an author, a timestamp and a pointer to its parent commit. Commits are identified by a long hash such as `3f9c2ab...`, and the first seven characters are usually enough to name one. Because every commit points to its parent, the history is a chain you can walk backwards.

Git stores content efficiently by keeping identical files once, so snapshots are cheap. Think of the history as a list of save points you can return to or compare at any time.

Create a repository and commit

Tell Git who you are once, then start a repository:

```
git config --global user.name "Asha Rao"
git config --global user.email "asha@example.com"
git config --global init.defaultBranch main

mkdir backup-tools && cd backup-tools
git init
echo "#!/bin/sh" > backup.sh
git status
git add backup.sh
git commit -m "Add backup script skeleton"
```

`git init` creates a hidden `.git` directory that holds the whole history; delete it and the project is just files again. `git status` is the command you will run most, because it tells you what is changed, what is staged and what is untracked. The commit includes only what you staged with `git add`, not everything in the folder.

The three areas

Files move through three places. The **working tree** is the files you edit. The **staging area** (also called the index) is what will go into the next commit. The **repository** is the saved history. `git add` copies a change from the working tree into the staging area, and `git commit` turns the staged set into a permanent snapshot. This two-step design lets you edit ten files and commit them as two clean, separate changes: stage the first group, commit, then stage the rest.

`git add -p` lets you stage individual pieces of a file, which is how experienced people keep commits focused.

MARGA by EknathaLabs marga.eknathalabs.com

Read the history

```
git log --oneline --graph --decorate -10
git show 3f9c2ab
git diff                 # working tree vs staging
git diff --staged        # staging vs last commit
git log -p -- backup.sh  # every change to one file
```

A good commit does one thing and its message says why. Write a short summary line in the imperative ("Fix retry loop in backup.sh"), leave a blank line, then explain the reason if it is not obvious. Months later, `git log` and `git blame` are your documentation of why the code looks the way it does, so a message like "update" wastes that chance.

Leave things out with .gitignore

Some files do not belong in history: compiled output, virtual environments, logs, editor settings and above all secrets. List patterns in a `.gitignore` file at the top of the repository:

```
__pycache__/
*.log
.venv/
dist/
.env
terraform.tfstate*
```

Ignore rules only affect files Git is not yet tracking. If a file was committed before you ignored it, it stays tracked until you remove it from the index with `git rm --cached FILE`. Add the `.gitignore` in your very first commit, because it is much easier to avoid a bad file than to remove it from history later.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Your first repository: snapshots, commits and... fits together

Part	What it does	What goes wrong
Snapshots, not copies of files	Git is a distributed version control system.	Committing generated files, logs or secrets.
Create a repository and commit	Tell Git who you are once, then start a repository.	Making one huge commit that mixes unrelated changes.
The three areas	Files move through three places. The working tree is the files you edit.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Read the history	A good commit does one thing and its message says why.	Writing messages like "update" or "fix stuff".
Leave things out with .gitignore	Some files do not belong in history: compiled output, virtual environments, logs, editor settings and above all secrets.	Skipping git status before committing.

Flow: Snapshots, not copies of files to Create a repository and commit / Create a repository and commit to The three areas / The three areas to Read the history / Read the history to Leave things out with .gitignore

Incident: Two gigabytes of build output

Incident: *A new teammate cloned the deployment repository and waited twenty minutes for a download that should have been seconds.*

```
$ git clone ssh://code.example.com/ops/deploy.git
Receiving objects: 100% (48211/48211), 1.92 GiB | 2.1 MiB/s
$ git count-objects -vH | grep size-pack
size-pack: 1.88 GiB
$ git log --oneline --stat -3 | head
9ad41e0 Add release bundles
 release/app-1.4.tar.gz | Bin 0 -> 412308120 bytes
```

MARGA by EknathaLabs
marga.eknathalabs.com

Someone had committed compiled release bundles several times, and because Git keeps every version forever, each bundle stayed in history even after being deleted in later commits. Removing them from history required rewriting the repository (covered in the lesson on secrets and history). The lasting fix was a .gitignore for the dist directory and a rule that release artifacts go to an artifact store, not to source control.

Common mistakes

- Committing generated files, logs or secrets
- Writing messages like "update" or "fix stuff"
- Making one huge commit that mixes unrelated changes
- Skipping git status before committing

Check yourself

What is staged by git add?

Answer: The changes that will be included in the next commit

Where is the full history stored?

Answer: In the .git directory of each clone

Which command shows what is staged compared with the last commit?

Answer: git diff --staged

A file was committed before being added to .gitignore. What happens?

Answer: It stays tracked until you remove it from the index with git rm --cached

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Undoing changes safely: restore, amend, revert, reset and stash

13 minutes. Pick the right undo for the situation: discard edits, unstage, fix the last commit, reverse a shared commit, or park work with stash.

Everyone makes mistakes in Git: a wrong file staged, a typo in the last commit message, a change that should not have been committed at all. The fear of making them is what keeps beginners away from the tool. Git has a safe undo for almost every case, but they are different commands for different situations, and the dangerous ones are the ones that rewrite history others may already have.

What you will learn

- Discard working tree edits and unstage files with git restore
- Fix the most recent commit with git commit --amend
- Undo a commit that others already have with git revert
- Understand soft, mixed and hard reset, and park work with git stash

Undo before you commit

Two modern commands cover the common cases:

```
git restore backup.sh              # throw away edits in the working tree (cannot be undone)
git restore --staged backup.sh     # unstage, but keep the edits
git restore --source=HEAD~2 backup.sh   # bring back the file as it was two commits ago
```

The first one is destructive: edits that were never committed are gone, because Git never saw them as a snapshot. Read `git status` first; it prints the correct command as a hint. Older tutorials use `git checkout -- FILE` and `git reset HEAD FILE` for the same jobs.

Fix the last commit

If you forgot a file or made a typo in the message and have **not pushed**, amend:

```
git add forgotten.sh
git commit --amend          # opens the editor to change the message too
git commit --amend --no-edit # keep the message, add the staged changes
```

Amend does not edit the old commit. It creates a new commit with a new hash and moves the branch to it, so any copy that someone else already fetched no longer matches. That is why rewriting history is fine for local work and a problem for shared work.

Undo something that is already shared

When a commit is already on a shared branch, do not rewrite history. Add a new commit that reverses it:

```
git revert 3f9c2ab          # creates a new commit with the opposite change
git revert -m 1 7d1e0aa     # to reverse a merge commit, choose the mainline parent
```

The history stays honest ("this was changed, then changed back") and everyone can pull the fix normally. Revert is the right tool for production rollbacks of code, and the revert commit should say why.

Reset: moving the branch

`git reset` moves the current branch pointer to another commit. The mode decides what happens to your files:

Mode	Branch moves	Staging area	Working tree
`--soft`	yes	kept	kept
`--mixed` (default)	yes	reset	kept
`--hard`	yes	reset	**overwritten**

MARGA by EknathaLabs marga.eknathalabs.com

```
git reset --soft HEAD~1     # undo the last commit, keep everything staged
git reset --hard origin/main # make my branch exactly match the remote (local work is lost)
```

Use `--hard` only when you are sure there is nothing uncommitted you want. Even after a hard reset, committed work is usually recoverable through the reflog (see the lesson on debugging Git).

Park work with stash

You are in the middle of a change and need to switch tasks:

```
git stash push -m "half-done retry logic"
git switch hotfix/disk-alert
# ... fix, commit, switch back ...
git stash list
git stash pop        # reapply and drop the newest stash
```

By default stash saves tracked changes; add `-u` to include untracked files. A stash is local and easy to forget, so treat it as a short-term parking space and commit work in progress on a branch if it will sit for days.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Undoing changes safely: restore, amend, revert... fits together

Part	What it does	What goes wrong
Undo before you commit	Two modern commands cover the common cases.	Amending or force-pushing commits that others already pulled.
Fix the last commit	If you forgot a file or made a typo in the message and have not pushed, amend.	Leaving work in a stash for weeks.
Undo something that is already	When a commit is already on a shared branch, do not rewrite history.	Reverting a merge without choosing the mainline parent.
Reset: moving the branch	git reset moves the current branch pointer to another commit.	Using reset --hard casually.
Park work with stash	You are in the middle of a change and need to switch tasks.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Undo before you commit to Fix the last commit / Fix the last commit to Undo something that is already / Undo something that is already to Reset: moving the branch / Reset: moving the branch to Park work with stash

Incident: The hard reset that erased a day of work

Incident: *An engineer tried to "clean up" before a meeting and ran a hard reset on the wrong branch.*

```
$ git status -s
 M deploy/values.yaml
 M deploy/ingress.yaml
$ git reset --hard origin/main
HEAD is now at 41ac9d2 Merge pull request 482
$ git status -s
$ git reflog | head -3
41ac9d2 HEAD@{0}: reset: moving to origin/main
7be3f10 HEAD@{1}: commit: Tune ingress timeouts
```

The two uncommitted files were lost for good, because Git had never saved them. The one commit was recovered from the reflog by checking out 7be3f10 on a new branch. The team added a habit of running git status and git stash before any reset, and shell aliases that print what a hard reset will discard.

Common mistakes

- Using reset --hard casually
- Amending or force-pushing commits that others already pulled
- Reverting a merge without choosing the mainline parent

MARGA by EknathaLabs marga.eknathalabs.com

- Leaving work in a stash for weeks

Check yourself

Which command discards uncommitted edits to a file?

Answer: git restore FILE

Why use git revert for a shared commit?

Answer: It adds a new commit and does not rewrite history that others have

What does git reset --soft HEAD~1 do?

Answer: Moves the branch back one commit and keeps the changes staged

What does git stash push save by default?

Answer: Changes to tracked files

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Branches and merging

13 minutes. Create branches for each piece of work, merge them back with fast-forward or merge commits, and resolve conflicts without panic.

On a team, two people change the same project at the same time. Without a way to separate their work, they overwrite each other. Branches are Git's answer: cheap, instant parallel lines of history that you combine later. Because a branch is only a pointer to a commit, creating one costs nothing, and the habit of one branch per task keeps `main` stable.

What you will learn

- What a branch really is: a movable pointer to a commit
- Create and switch branches, and see where HEAD is
- Merge with fast-forward and with merge commits
- Resolve a merge conflict step by step

A branch is a pointer

A branch is a small file containing the hash of one commit. When you commit on a branch, the pointer moves forward to the new commit. `HEAD` is the pointer to the branch you are on. Because nothing is copied, branching is instant even in huge repositories.

```
git branch                      # list local branches
git switch -c feature/retry     # create and move to a new branch
git log --oneline --graph --all --decorate
git switch main
git branch -d feature/retry     # delete a merged branch
```

`git switch` and `git restore` replace the older, overloaded `git checkout`. Branch names with a slash (`feature/`, `fix/`) group related branches and many teams add a ticket number.

Merging

Merging combines the work of another branch into the current one:

```
git switch main
git merge feature/retry
```

If `main` has not moved since you branched, Git simply moves the pointer forward: a **fast-forward**, with no new commit. If both branches have new commits, Git creates a **merge commit** with two parents that ties the lines together. Use `git merge --no-ff` to always record a merge commit, which keeps the fact that a group of commits belonged together. Before merging, `git diff main...feature/retry` shows what the branch would add.

Conflicts

A **conflict** happens when both branches changed the same lines. Git stops and marks the files:

```
<<<<<<< HEAD
retries=3
=======
retries=5
>>>>>>> feature/retry
```

The part between `<<<<<<<` and `=======` is your side, the rest is theirs. Resolve it by editing the file to the correct final content, removing all marker lines, then:

```
git status                # lists "both modified" files
git add config/backup.conf
git commit                # concludes the merge
# or abandon it:
git merge --abort
```

MARGA by EknathaLabs
marga.eknathalabs.com

Do not choose a side blindly. Ask the author, run the tests and read the surrounding code, because a merge can be textually clean and still wrong, for example when one branch renames a function that the other starts to use.

Keeping branches healthy

Short-lived branches conflict less. Merge `main` into your branch (or rebase, see the next lesson) often, so differences stay small. Delete branches after they are merged. Avoid long-running branches that drift for months: the longer a branch lives, the more painful the merge becomes. Use `git merge --no-commit --no-ff other` to preview a merge, and a tool such as `git mergetool` or your editor's three-way view when conflicts are large.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Branches and merging fits together

Part	What it does	What goes wrong
A branch is a pointer	A branch is a small file containing the hash of one commit.	Long-lived branches that drift.
Merging	Merging combines the work of another branch into the current one.	Resolving conflicts without understanding both sides.
Conflicts	A conflict happens when both branches changed the same lines.	Leaving conflict markers in a file and committing it.
Keeping branches healthy	Short-lived branches conflict less. Merge main into your branch (or rebase, see the next lesson) often, so differences stay small.	Merging without running tests on the result.

Flow: A branch is a pointer to Merging / Merging to Conflicts / Conflicts to Keeping branches healthy

Incident: The conflict that was resolved wrongly

Incident: *A deploy script merged cleanly and failed in production an hour later.*

```
$ git merge release/2.3
Auto-merging deploy.sh
Merge made by the 'ort' strategy.
$ ./deploy.sh staging
ERROR: unknown option --region-id
$ git log -3 --oneline -- deploy.sh
8c0aa21 Rename --region-id to --region
4be7710 Add --region-id handling for blue/green
```

One branch had renamed an option while another had added a call using the old name. Git saw changes in different lines and merged without a conflict, but the combined result was broken. The team started running the test suite on the merge result in the pipeline, not only on each branch, and added a check that runs the script with the new options in staging before merging to main.

Common mistakes

- Long-lived branches that drift
- Resolving conflicts without understanding both sides
- Leaving conflict markers in a file and committing it
- Merging without running tests on the result

Check yourself

What is a Git branch?

Answer: A movable pointer to a commit

MARGA by EknathaLabs marga.eknathalabs.com

When does a fast-forward merge happen?

Answer: When the target branch has no new commits since the branch point

What do the lines <<<<<<< and >>>>>>> mean?

Answer: They mark the two conflicting versions that you must resolve

How do you abandon a merge in progress?

Answer: git merge --abort

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Rebase and a clean history

13 minutes. Use rebase to keep a linear history, tidy your own commits with interactive rebase, and know the rule for when not to rebase.

Merging is safe but the history of a busy project fills with merge commits and "fix typo" commits that make it hard to read. Rebase lets you replay your work on top of the latest main and tidy it before others review it. It is powerful and it is the command that has caused the most Git horror stories, because it rewrites history. The rules are simple and worth learning properly.

What you will learn

- What rebase does and how it differs from merge
- Update a feature branch with git rebase main
- Tidy local commits with interactive rebase (squash, reword, fixup)
- The golden rule: never rebase commits that others have built on

Rebase replays your commits

Suppose `main` moved on while you worked on `feature`. A merge ties the two lines with a merge commit. A **rebase** instead takes your commits, sets them aside, moves your branch to the tip of `main`, and replays your commits one by one on top. The result looks as if you had started your work from the latest `main`.

```
git switch feature/retry
git fetch origin
git rebase origin/main
```

The replayed commits are **new commits** with new hashes, though they have the same changes and messages. If a replayed commit conflicts, Git stops; resolve the files, `git add` them, and run `git rebase --continue`. `git rebase --abort` returns you to where you started.

Interactive rebase

Before asking for a review, clean up your own work:

```
git rebase -i HEAD~4
```

Git opens a list of the last four commits:

```
pick 1a2b3c4 Add retry loop
pick 5d6e7f8 Fix typo
pick 9a8b7c6 Add tests for retry
pick 0f1e2d3 Fix test name
```

Change the word at the start of each line: `reword` to edit a message, `squash` to merge into the previous commit and combine messages, `fixup` to merge and discard the message, `drop` to remove a commit, or reorder lines to reorder commits. Save and close, and Git rewrites the history. A small number of meaningful commits ("Add retry loop", "Add tests") is far easier to review than ten snapshots of your day.

The shortcut `git commit --fixup HASH` followed by `git rebase -i --autosquash` marks and moves fixes automatically.

The golden rule

Never rebase commits that exist in a shared branch others have pulled. Rebase creates new commits, so a teammate's copy of the old ones no longer matches, and their next pull produces duplicates and confusing conflicts. It is safe to rebase your own, unpushed work or a branch only you use. If you rebased a branch you had already pushed, the push is rejected, and you need `git push --force-with-lease`, which refuses to overwrite commits you have not seen. Use it only on your own feature branches, never on `main`.

MARGA by EknathaLabs marga.eknathalabs.com

Merge or rebase?

Both are valid, and teams choose a policy. Common choices: rebase your feature branch on `main` to keep it current, then merge with a pull request using **squash** (one commit per feature) or **merge commit** (keeps the branch history). Linear history makes `git bisect` and `git log` easier to read; merge commits preserve exactly what happened. Pick one policy per repository, write it in the contributing guide, and let the platform enforce it.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Rebase and a clean history fits together

Part	What it does	What goes wrong
Rebase replays your commits	Suppose main moved on while you worked on feature.	Using git push --force instead of --force-with-lease.
Interactive rebase	Before asking for a review, clean up your own work.	Squashing everything into a single huge commit.
The golden rule	Never rebase commits that exist in a shared branch others have pulled.	Rebasing a branch that others use.
Merge or rebase?	Both are valid, and teams choose a policy.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Rebase replays your commits to Interactive rebase / Interactive rebase to The golden rule / The golden rule to Merge or rebase?

Incident: The force push that removed a colleague's commit

Incident: *Two engineers worked on the same branch and one rebased and force-pushed.*

```
$ git push --force origin feature/alerts
+ 4e2fa1b...d19c07a feature/alerts -> feature/alerts (forced update)
$ git log --oneline origin/feature/alerts -3
d19c07a Tune alert thresholds
$ # Priya's commit "Add runbook link" is no longer on the branch
```

The force push replaced the branch and dropped a commit that the colleague had pushed a few minutes earlier. The commit was recovered from the colleague's local clone and the reflog on the server. The team switched to git push --force-with-lease, which would have refused, and agreed that shared branches are never rebased.

Common mistakes

- Rebasing a branch that others use
- Using git push --force instead of --force-with-lease
- Squashing everything into a single huge commit
- Rebasing in the middle of a conflict without understanding the state

Check yourself

What does git rebase main do on a feature branch?

Answer: Replays the branch commits on top of the latest main as new commits

Which interactive rebase keyword combines a commit into the previous one and discards its message?

Answer: fixup

When is it unsafe to rebase?

Answer: When others have already pulled and built on those commits

MARGA by EknathaLabs
marga.eknathalabs.com

Why prefer --force-with-lease to --force?

Answer: It refuses to overwrite remote commits you have not seen

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Remotes: clone, fetch, pull and push

13 minutes. Work with a shared repository: clone it, see what changed with fetch, update with pull, publish with push, and understand why a push is rejected.

A repository on your laptop is only useful to others if it is connected to a shared one. A remote is a named link to another copy of the repository, usually on a server or a code-sharing site. Most daily Git confusion ("why is my push rejected?", "what did pull just do?") comes from not seeing that your branches and the remote's branches are separate things that you synchronize on purpose.

What you will learn

- Clone and inspect remotes, and what origin means
- Remote-tracking branches such as origin/main
- The difference between fetch and pull
- Push, upstream branches, rejected pushes and safe forcing

Clone and remotes

```
git clone git@code.example.com:ops/deploy.git
cd deploy
git remote -v
git branch -vv
```

`git clone` copies the full history and creates a remote named `origin` pointing to where you cloned from. You can have more than one remote, for example `origin` for your fork and `upstream` for the original project. `git branch -vv` shows each local branch, the remote branch it tracks, and whether you are ahead or behind.

A remote can use SSH (`git@host:path`) or HTTPS (`https://host/path`). SSH uses your key pair and avoids typing passwords; HTTPS usually uses a token stored by a credential helper. Never put a token in the URL of a remote, because it ends up in `.git/config` and shell history.

Remote-tracking branches

After a clone, Git keeps a read-only copy of the remote's branches as `origin/main`, `origin/feature-x` and so on. They only move when you talk to the remote. This is why your local `main` and `origin/main` can differ: one is your work, the other is your last known picture of the server.

```
git fetch origin          # update origin/* from the server, change nothing else
git log --oneline main..origin/main   # commits on the remote that I do not have
git log --oneline origin/main..main   # my commits not yet pushed
```

Fetch versus pull

`git fetch` downloads new commits and updates remote-tracking branches. It never touches your working tree, so it is always safe. `git pull` is `fetch` followed by an integration step: by default a merge, or a rebase if you ask for it.

```
git pull --ff-only      # fail instead of creating a surprise merge commit
git pull --rebase       # replay my local commits on top
git config --global pull.ff only   # make the safe behaviour the default
```

A habit that avoids surprises: `git fetch`, look at what arrived with `git log`, then integrate. If you see an unexpected merge commit after a pull, it means you and the remote both had new commits.

Push and rejected pushes

```
git push -u origin feature/retry    # first push: create the branch and set its upstream
git push                            # later pushes need no arguments
```

A push succeeds only if it is a fast-forward of the remote branch. If someone else pushed first, you see `! [rejected] (non-fast-forward)`. Do not reach for `--force`. Fetch, integrate their commits (rebase or merge), run your tests and push again. If you really rewrote your own feature branch, use `git push`

MARGA by EknathaLabs marga.eknathalabs.com

`--force-with-lease`, which fails if the remote has commits you have not seen. Many teams protect `main` so that nobody, including administrators, can force-push to it.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Remotes: clone, fetch, pull and push fits together

Part	What it does	What goes wrong
Clone and remotes	git clone copies the full history and creates a remote named origin pointing to where you cloned from.	Pulling without knowing whether it merges or rebases.
Remote-tracking branches	After a clone, Git keeps a read-only copy of the remote's branches as origin/main, origin/feature-x and so on.	Putting access tokens in remote URLs.
Fetch versus pull	git fetch downloads new commits and updates remote-tracking branches.	Forgetting to set the upstream so push and pull need arguments.
Push and rejected pushes	A push succeeds only if it is a fast-forward of the remote branch.	Using --force to get past a rejected push.

Flow: Clone and remotes to Remote-tracking branches / Remote-tracking branches to Fetch versus pull / Fetch versus pull to Push and rejected pushes

Incident: The push that overwrote a release branch

Incident: *A hurried engineer saw a rejected push and fixed it the quick way.*

```
$ git push origin release/3.2
 ! [rejected]        release/3.2 -> release/3.2 (non-fast-forward)
$ git push --force origin release/3.2
 + a91c3e0...22d7f10 release/3.2 -> release/3.2 (forced update)
$ git log --oneline origin/release/3.2 -2
22d7f10 Bump version to 3.2.4
```

The force push removed two security fix commits from the release branch that a colleague had pushed an hour earlier. The commits were found in the colleague's clone and restored. The repository now blocks force pushes on release/* and main, and the contributing guide tells people to fetch and rebase when a push is rejected.

Common mistakes

- Pulling without knowing whether it merges or rebases
- Using --force to get past a rejected push
- Putting access tokens in remote URLs
- Forgetting to set the upstream so push and pull need arguments

Check yourself

What does git fetch change?

Answer: Remote-tracking branches such as origin/main, not your working tree

What is git pull?

Answer: A fetch followed by a merge or rebase

Why is a push rejected as non-fast-forward?

Answer: The remote branch has commits that you do not have

What does git push -u origin BRANCH do?

Answer: Pushes the branch and sets it as the upstream of the local branch

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Team workflows: branching strategies, pull requests and reviews

14 minutes. Choose a branching strategy, run small reviewed pull requests, and protect main with required checks.

Git does not tell you how a team should work. Without agreed rules, five people push to main in five styles, reviews are rare, and releases are tense. A workflow is a small set of rules about branches, reviews and merges. The best ones are boring: short branches, small changes, automatic checks, and a main branch that is always releasable.

What you will learn

- Compare trunk-based development, feature branches and release branches
- Write pull requests that are easy to review
- Protect main with required reviews and status checks
- Use CODEOWNERS and clear commit messages

Three common strategies

Trunk-based development: everyone merges small changes to `main` at least daily, often behind feature flags. It needs good tests and a fast pipeline, and gives the least merge pain. **Feature branches** (a simple flow that works with any hosting): one short branch per change, reviewed through a pull request, then merged to `main` and deployed. **Release branches** (as in Gitflow): long-lived `develop` and `release/*` branches, useful when you must support several released versions at once, but slower and more complex. For web services and infrastructure code, trunk-based or short feature branches usually fit best; for installed software with several supported versions, release branches earn their cost.

A pull request people can review

A pull request (also called a merge request) proposes merging one branch into another and is the place for discussion, review and automated checks. Good habits:

- Keep it small: a few hundred changed lines at most. Large changes get skimmed, not reviewed.
- Write what and why in the description, how you tested it, and link the ticket.
- Separate refactoring from behaviour changes.
- Answer comments with a commit or an explanation, and re-request review.

Reviewers should read for correctness, safety (what happens when this fails?), clarity and tests, and say which comments are blocking and which are suggestions. For infrastructure code, attach the plan output, because the diff of code does not show what will be created or destroyed.

Protect the main branch

Hosting platforms let you set rules on `main` that nobody can bypass by accident:

```
Require a pull request before merging
Require at least 1 approving review (2 for production config)
Require status checks: lint, unit tests, security scan
Require branches to be up to date before merging
Block force pushes and branch deletion
Require signed commits (optional)
```

The checks must run on the pull request, and the merge button stays disabled until they pass. A `CODEOWNERS` file maps paths to teams, so a change under `deploy/prod/` automatically requests review from the platform team:

```
/deploy/prod/   @acme/platform-team
*.tf            @acme/infra-reviewers
```

MARGA by EknathaLabs
marga.eknathalabs.com

Commit and merge conventions

Pick one merge style (squash, rebase or merge commit) and use it consistently. Many teams adopt a message convention such as `type(scope): summary` (for example `fix(backup): retry on timeout`), because tools can then build changelogs and decide version numbers. Keep the first line under about 72 characters. The point of all these rules is the same: history that a stranger can read at 03:00 during an incident.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Team workflows: branching strategies, pull... fits together

Part	What it does	What goes wrong
Three common strategies	Trunk-based development: everyone merges small changes to main at least daily, often behind feature flags.	Long-lived branches with unclear owners.
A pull request people can review	A pull request (also called a merge request) proposes merging one branch into another and is the place for discussion, review and automated checks.	Required checks that are flaky, so people bypass them.
Protect the main branch	Hosting platforms let you set rules on main that nobody can bypass by accident.	Allowing direct pushes to main.
Commit and merge conventions	Pick one merge style (squash, rebase or merge commit) and use it consistently.	Huge pull requests.

Flow: Three common strategies to A pull request people can review / A pull request people can review to Protect the main branch / Protect the main branch to Commit and merge conventions

Incident: The unreviewed hotfix

Incident: *A late-night fix was pushed straight to main because the rule was only a convention.*

```
$ git log --oneline -3 main
7fa90cd Hotfix: raise db pool size
$ git log -1 --format=%an,%cd 7fa90cd
sam, Thu Jan 5 23:48
$ curl -s https://shop.example.com/health
{"db":"error: too many connections"}
```

The fix raised the connection pool beyond what the database allowed and took the service down at the next restart. Nobody had reviewed it because direct pushes to main were possible. After the incident the team enabled branch protection with required review and checks, and added an expedited path: a pull request that needs one reviewer and runs the fast checks, so an urgent fix is still reviewed.

Common mistakes

- Allowing direct pushes to main
- Huge pull requests
- Required checks that are flaky, so people bypass them
- Long-lived branches with unclear owners

Check yourself

What does trunk-based development ask of the team?

Answer: Small changes merged to main frequently, supported by tests and a fast pipeline

What is a pull request for?

Answer: Proposing a merge and holding review, discussion and automated checks

MARGA by EknathaLabs
marga.eknathalabs.com

What does CODEOWNERS do?

Answer: Maps paths to the people or teams who must review changes

Which setting prevents history loss on main?

Answer: Blocking force pushes and branch deletion

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Tags, releases and version numbers

12 minutes. Mark releases with annotated tags, number them with semantic versioning, and handle hotfixes and changelogs.

In production, someone asks, "which exact code is running?" A branch name is no answer, because branches move. A **tag** is a permanent name for one commit, and a version number on a tag is how teams, pipelines and customers talk about releases. Getting tags and versions right makes rollbacks, audits and incident analysis much simpler.

What you will learn

- Lightweight and annotated tags, and why releases use annotated ones
- Semantic versioning: MAJOR.MINOR.PATCH
- Release branches and hotfixes
- Changelogs and finding what is in a release

Tags

A tag is a fixed name for a commit. A **lightweight** tag is only a pointer. An **annotated** tag is a full object with a tagger, date and message, and can be signed. Use annotated tags for releases.

```
git tag -a v1.4.0 -m "Release 1.4.0: add retry to backup job"
git tag -s v1.4.1 -m "Hotfix, signed"     # signed with your key
git push origin v1.4.0                      # tags are not pushed by default
git push origin --tags
git show v1.4.0
git describe --tags                         # e.g. v1.4.0-12-g3f9c2ab
```

`git describe` turns any commit into a readable name: the last tag, how many commits since, and the short hash. Many pipelines use it to stamp the version into builds. Do not move or delete a published tag; deployed artifacts already refer to it. If a release is wrong, publish a new version.

Semantic versioning

Semantic versioning (SemVer) uses `MAJOR.MINOR.PATCH`:

- **PATCH** for backward-compatible bug fixes (1.4.0 to 1.4.1)
- **MINOR** for backward-compatible new features (1.4.1 to 1.5.0)
- **MAJOR** for breaking changes (1.5.0 to 2.0.0)

A pre-release is written `2.0.0-rc.1`. The numbers are a promise to the people who depend on you: a minor upgrade should not break them. Services that nobody installs themselves sometimes use dates (`2026.01.05`) or build numbers; choose one scheme and write it down.

Releases and hotfixes

A simple flow: develop on short branches, merge to `main`, tag the release commit, build the artifact from that tag, and deploy the same artifact everywhere. If you need to fix an old version, branch from its tag:

```
git switch -c hotfix/1.4 v1.4.0
# fix, commit, then
git tag -a v1.4.1 -m "Fix backup path"
git switch main && git cherry-pick HASH   # bring the fix forward
```

`git cherry-pick` copies one commit onto another branch. Always make sure the fix also reaches `main`, or the bug returns in the next release.

Changelogs and questions you can answer

Keep a human-written changelog that lists notable changes per version under headings such as Added, Changed, Fixed and Security. Generating it from commit messages works if messages are disciplined. Tags let you answer production questions quickly:

MARGA by EknathaLabs marga.eknathalabs.com

```
git log --oneline v1.3.0..v1.4.0       # what is in this release
git tag --contains 3f9c2ab              # which releases contain this fix
git diff v1.4.0 v1.4.1 --stat           # what the hotfix changed
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Tags, releases and version numbers fits together

Part	What it does	What goes wrong
Tags	A tag is a fixed name for a commit. A lightweight tag is only a pointer.	Lightweight tags for releases.
Semantic versioning	Semantic versioning (SemVer) uses MAJOR.MINOR.PATCH.	Breaking changes in a minor version.
Releases and hotfixes	A simple flow: develop on short branches, merge to main, tag the release commit, build the artifact from that tag, and deploy the same artifact everywhere.	Hotfixes that never reach main.
Changelogs and questions you can	Keep a human-written changelog that lists notable changes per version under headings such as Added, Changed, Fixed and Security.	Moving or deleting published tags.

Flow: Tags to Semantic versioning / Semantic versioning to Releases and hotfixes / Releases and hotfixes to Changelogs and questions you can

Incident: The tag that moved

Incident: *Two builds both said version 2.1.0, and they behaved differently.*

```
$ git rev-parse v2.1.0
9e0b7c1
$ git ls-remote --tags origin v2.1.0
4c18aa2    refs/tags/v2.1.0
$ docker image inspect shop:2.1.0 --format "{{index .Config.Labels \"commit\"}}"
9e0b7c1
```

A developer had deleted and recreated the tag v2.1.0 on a different commit after a late fix. Production still ran the image built from the first commit, while the new tag pointed to different code. The team banned moving tags (the server now rejects tag updates), started stamping the commit hash into every image, and decided that a mistake means releasing 2.1.1.

Common mistakes

- Moving or deleting published tags
- Lightweight tags for releases
- Breaking changes in a minor version
- Hotfixes that never reach main

Check yourself

Which tag type should releases use?

Answer: Annotated tags

Under SemVer, which change increments MAJOR?

Answer: A backward-incompatible change

Are tags pushed by git push by default?

Answer: No, push them explicitly or with --tags

What does git tag --contains HASH show?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Which tags include that commit

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Hooks, signed commits and secrets in history

14 minutes. Catch mistakes with hooks, prove who made a commit with signing, and respond correctly when a secret has been committed.

A developer commits a cloud access key by accident. Deleting the file in the next commit does not help, because Git keeps the old version in history and anyone with a clone has it. This lesson covers prevention (hooks and scanning), proof of authorship (signing) and the correct, ordered response when prevention fails. The order matters: revoke first, clean later.

What you will learn

- Client-side hooks and the pre-commit approach
- Server-side protection and secret scanning
- Signed commits and tags
- Responding to a leaked secret: revoke, rewrite, notify

Hooks

Hooks are scripts that Git runs at certain moments. They live in `.git/hooks`, which is not shared by clone, so teams manage them with a tool or a shared directory.

```
# .git/hooks/pre-commit  (must be executable)
#!/bin/sh
if git diff --cached --name-only | grep -E "\.env$|id_rsa$"; then
  echo "Refusing to commit a secret file"; exit 1
fi
```

Common hooks: `pre-commit` (lint, format, scan), `commit-msg` (check the message format) and `pre-push` (run fast tests). The `pre-commit` framework lets you list hooks in a `.pre-commit-config.yaml` file that everyone installs with one command. Hooks can be skipped with `--no-verify`, so they help honest people, and the real enforcement belongs on the server and in the pipeline.

Server-side protection

Run the same checks in the pipeline on every pull request, and turn on **push protection** or secret scanning on the hosting platform so that known token formats are blocked or reported. Add a scanner such as a secrets-detection tool to the pipeline, scan the full history periodically, and treat any finding as an incident. Keep real secrets in a secret manager and use short-lived credentials, so a leak is less damaging.

Signed commits

Anyone can set `user.name` to anything, so the author line proves nothing. A signature proves the commit came from the holder of a key. Git can sign with GPG or, more simply, with an SSH key:

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git log --show-signature -1
```

The hosting platform shows a "verified" badge when it knows your public key, and a branch rule can require signed commits. Signing matters most for release tags and for repositories that deploy automatically.

A secret was committed: the order of actions

1. **Revoke and rotate the secret immediately.** Assume it is already copied. Rewriting history without revoking leaves a working credential in the wild.
2. Check logs for use of the secret during the exposure.

MARGA by EknathaLabs marga.eknathalabs.com

3 Remove it from history if needed, using a rewriting tool such as `git filter-repo`:

```
git filter-repo --path config/prod.env --invert-paths
git push --force-with-lease --all
```

1 Ask everyone to re-clone or reset, because old clones and forks still contain it. Ask the hosting provider to purge cached views of the old commits.

2 Add a prevention (hook, push protection, ignore rule) and write a short review of how it happened.

Cleaning history is secondary; the credential must be dead before you start.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Hooks, signed commits and secrets in history fits together

Part	What it does	What goes wrong
Hooks	Hooks are scripts that Git runs at certain moments.	Rotating nothing and only deleting the file.
Server-side protection	Run the same checks in the pipeline on every pull request, and turn on push protection or secret scanning on the hosting platform so that known token formats are blocked...	Relying on a local hook that can be skipped.
Signed commits	Anyone can set user.name to anything, so the author line proves nothing.	Using shared long-lived keys in repositories.
A secret was committed: the order	Revoke and rotate the secret immediately.	Rewriting history without telling the team.

Flow: Hooks to Server-side protection / Server-side protection to Signed commits / Signed commits to A secret was committed: the order

Incident: The key in the config file

Incident: *A scanner raised an alert on a repository that had been public for three days.*

```
$ git log --all --oneline -S"AKIA" -- config/
5b2c9de Add backup settings
$ aws cloudtrail lookup-events --lookup-attributes AttributeKey=AccessKeyId,AttributeValue=AKIAEXA \
    MPLE --max-results 3
RunInstances   2026-01-04T02:11:09Z  region ap-south-1
```

An access key was in a settings file, and someone had used it to start compute instances. The team revoked the key within minutes, terminated the instances and then rewrote the history. They installed push protection, added a pre-commit scan, moved secrets to a secret manager and made the CI role assumable with short-lived credentials so no static key exists to leak.

Common mistakes

- Rotating nothing and only deleting the file
- Relying on a local hook that can be skipped
- Rewriting history without telling the team
- Using shared long-lived keys in repositories

Check yourself

What is the first step after a secret is committed?

Answer: Revoke and rotate the secret

Why are local hooks not enough?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: They can be skipped or not installed, so enforcement belongs on the server and in the pipeline

What does a signed commit prove?

Answer: That the commit was made by the holder of the signing key

Which tool can rewrite history to remove a file everywhere?

Answer: git filter-repo

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Large repositories, submodules and worktrees

13 minutes. Keep big repositories fast with shallow and partial clones, sparse checkout and LFS; know when to use submodules; work on several branches at once with worktrees.

Repositories grow. A project that cloned in seconds takes minutes after years of history, large binaries and many contributors. CI jobs clone on every run, so slow clones become slow pipelines and real money. Git has several tools to keep large repositories usable, and a few ways to combine repositories. Knowing which to pick, and which to avoid, saves many headaches.

What you will learn

- Shallow, partial and sparse clones for speed
- Large files with Git LFS, and when to keep binaries out
- Submodules and alternatives for sharing code
- Worktrees: several branches checked out at once

Clone less

Not every job needs the full history.

```
git clone --depth 1 URL                    # shallow: only the latest commit
git clone --filter=blob:none URL           # partial: fetch file contents on demand
git clone --filter=blob:none --sparse URL
cd repo && git sparse-checkout set services/payments docs
```

A **shallow clone** is ideal for CI builds that only need the current files, but commands that need history (`git describe`, `git log`, blame, some versioning tools) will not work, so deepen it with `git fetch --deepen=50` or `--unshallow`. A **partial clone** keeps all commits but downloads file contents only when needed. **Sparse checkout** limits the working tree to chosen folders, which is how huge monorepos stay manageable.

Large files

Git stores every version of every file. A 200 MB binary changed ten times is 2 GB of history. Options:

- Keep build artifacts and large data **out** of Git; use an artifact registry or object storage and record the version in a file.
- For binaries that must live with the code (design files, test data), use **Git LFS**: Git stores a small pointer, and the large content goes to a separate store.

```
git lfs install
git lfs track "*.psd"
git add .gitattributes
```

The tracking rules are saved in `.gitattributes`. Decide before the files are committed; moving existing large files to LFS needs a history rewrite. Run `git count-objects -vH` and `git rev-list --objects --all | sort -k2 | head` style checks, or a size report in CI, to notice growth early.

Submodules and alternatives

A **submodule** embeds another repository at a fixed commit inside yours:

```
git submodule add https://code.example.com/ops/shared-scripts.git vendor/scripts
git clone --recurse-submodules URL
git submodule update --init --recursive
```

The parent records an exact commit of the child, which is good for reproducibility and a common source of confusion: people forget to update the submodule, commit in a detached state, or clone without `--recurse-submodules` and get empty folders. Alternatives are a package manager (publish the shared code as a versioned package), `git subtree` (copies the code into your history), or a monorepo. Prefer

MARGA by EknathaLabs
marga.eknathalabs.com

packages for libraries; use submodules when you truly need a pinned copy of another repository.

Worktrees

Switching branches in the middle of work means stashing or committing. A **worktree** gives you another folder with a different branch checked out, sharing the same repository data:

```
git worktree add ../hotfix-1.4 hotfix/1.4
cd ../hotfix-1.4
# fix and test while the main folder keeps your feature work
git worktree list
git worktree remove ../hotfix-1.4
```

This is handy for urgent fixes, reviewing a pull request in a clean directory, and running long tests on one branch while you code on another. Housekeeping runs automatically, and `git maintenance start` can schedule background optimization for large repositories.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Large repositories, submodules and worktrees fits together

Part	What it does	What goes wrong
Clone less	Not every job needs the full history.	Cloning full history in every CI job.
Large files	Git stores every version of every file. A 200 MB binary changed ten times is 2 GB of history.	Committing large binaries without LFS or an artifact store.
Submodules and alternatives	A submodule embeds another repository at a fixed commit inside yours.	Forgetting --recurse-submodules and building with empty folders.
Worktrees	Switching branches in the middle of work means stashing or committing.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Clone less to Large files / Large files to Submodules and alternatives / Submodules and alternatives to Worktrees

Incident: The pipeline that cloned for eleven minutes

> **Incident:** *Builds became slower each month, and a pipeline run spent most of its time before the first build step.*

```
$ time git clone ssh://code.example.com/platform/mono.git
Receiving objects: 100% (3604211/3604211), 7.4 GiB
real 11m02s
$ time git clone --depth 1 --filter=blob:none ssh://code.example.com/platform/mono.git
real 0m19s
```

The job needed only the files of one service. The team changed the pipeline to use a shallow, partial clone with sparse checkout of the service folder and its shared libraries, which cut clone time to under 20 seconds. They also moved old release bundles out of the repository and added a size alert so growth is noticed early.

Common mistakes

- Committing large binaries without LFS or an artifact store
- Cloning full history in every CI job
- Forgetting --recurse-submodules and building with empty folders
- Editing submodule contents in a detached state and losing the commit

Check yourself

What does git clone --depth 1 do?

Answer: Downloads only the most recent commit

MARGA by EknathaLabs marga.eknathalabs.com

What is a good place for large build artifacts?

Answer: An artifact registry or object storage, not the Git history

A submodule folder is empty after cloning. What did you likely forget?

Answer: git clone --recurse-submodules (or git submodule update --init)

What is a worktree?

Answer: An additional working directory with another branch checked out from the same repository

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Debugging Git problems: reflog, bisect, blame and recovery

14 minutes. Recover lost commits with the reflog, find the commit that broke something with bisect and blame, and understand detached HEAD and other confusing states.

At some point Git will show you something frightening: "detached HEAD", a branch that vanished, a commit you cannot find, a build that broke between two releases. In nearly every case the data is still there. Git almost never deletes committed work immediately, and it also keeps a private journal of where your branches pointed. Knowing how to read that journal and how to search history turns panic into a routine investigation.

What you will learn

- Find and recover lost commits with git reflog
- Locate the commit that introduced a bug with git bisect
- Answer who changed this and why with blame and log -S
- Understand detached HEAD, wrong-branch commits and unfinished operations

Recover with the reflog

The **reflog** records every time `HEAD` or a branch moved on your machine: commits, resets, rebases, checkouts. It is local and expires after weeks, but it is the safety net for almost every "I lost my work" moment.

```
git reflog -15
# 41ac9d2 HEAD@{0}: reset: moving to origin/main
# 7be3f10 HEAD@{1}: commit: Tune ingress timeouts
git branch rescue 7be3f10      # give the lost commit a name again
git switch rescue
```

Any commit you can name by hash is safe once a branch points to it. This works for commits, not for uncommitted edits, which Git never saw. A deleted branch is also recoverable: find its last hash in the reflog and recreate it with `git branch NAME HASH`. For commits that are in no reflog at all, `git fsck --lost-found` lists unreachable objects.

Find the guilty commit with bisect

You know version 1.3 worked and the current one does not, with 200 commits between. `git bisect` does a binary search and needs only about eight steps:

```
git bisect start
git bisect bad                 # current commit is broken
git bisect good v1.3.0         # this one worked
# Git checks out a commit in the middle. Test it, then say good or bad.
git bisect good
git bisect bad
# ... until it prints: 3f9c2ab is the first bad commit
git bisect reset
```

Better still, automate it: `git bisect run ./test.sh`, where the script exits 0 for good and non-zero for bad. A small reproducible test is the most valuable thing to write during any regression hunt.

Who changed this, and why

```
git blame -L 40,60 deploy.sh        # who last changed each line
git log -S"max_retries" --oneline   # commits that added or removed this text
git log -G"retry.*=" -p -- deploy.sh
git log --follow -- scripts/backup.sh   # history across renames
git show 3f9c2ab                    # the full change and message
```

Blame shows the last commit to touch a line, which may only be a reformat. Use `git blame -w -C` to ignore whitespace and detect moved code, and follow the commit message and linked ticket to learn the reason.

MARGA by EknathaLabs
marga.eknathalabs.com

The aim is to understand why the code is the way it is before you change it, not to blame a person.

Confusing states

Detached HEAD: you checked out a commit or tag, not a branch. Commits made here belong to no branch. If you want to keep them, run `git switch -c new-branch` right away. **Committed on the wrong branch**: create a branch at the current commit with `git branch feature`, then move the wrong branch back with `git reset --hard origin/main` (after making sure the new branch exists). **Unfinished operations**: `git status` tells you when you are in the middle of a merge, rebase or cherry-pick and which command continues or aborts it. **Untracked clutter**: `git clean -n` previews what `git clean -fd` would delete. Read `git status` first; Git usually tells you the next safe command.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Debugging Git problems: reflog, bisect, blame and... fits together

Part	What it does	What goes wrong
Recover with the reflog	The reflog records every time HEAD or a branch moved on your machine: commits, resets, rebases, checkouts.	Committing in detached HEAD and losing the work.
Find the guilty commit with bisect	You know version 1.3 worked and the current one does not, with 200 commits between.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Who changed this, and why	Blame shows the last commit to touch a line, which may only be a reformat.	Using blame to assign fault instead of to learn the reason.
Confusing states	Detached HEAD: you checked out a commit or tag, not a branch.	Panicking and re-cloning instead of checking the reflog.

Flow: Recover with the reflog to Find the guilty commit with bisect / Find the guilty commit with bisect to Who changed this, and why / Who changed this, and why to Confusing states

Incident: Which release broke the report?

> **Incident:** *The monthly billing report began producing wrong totals, and nobody knew which of 214 commits since the last good release was responsible.*

```
$ git bisect start v2.8.0 v2.7.0
Bisecting: 107 revisions left to test after this (roughly 7 steps)
$ git bisect run ./check_totals.sh
running ./check_totals.sh
...
3d51f8a is the first bad commit
    Round currency in the aggregate query
```

A small script that compared the report with known totals was enough for git bisect run to find the commit in seven steps: a change that rounded values too early. The fix was a one-line change and the script became a regression test in the pipeline. Without bisect, the team would have read hundreds of diffs.

Common mistakes

- Panicking and re-cloning instead of checking the reflog
- Committing in detached HEAD and losing the work
- Using blame to assign fault instead of to learn the reason
- Running git clean -fd without the -n preview

Check yourself

Where do you find hashes of commits you lost after a reset?

Answer: In git reflog

MARGA by EknathaLabs
marga.eknathalabs.com

What does git bisect do?

Answer: Binary-searches history to find the first bad commit

You commit in detached HEAD. How do you keep the work?

Answer: Create a branch there: git switch -c NAME

Which command finds commits that added or removed a given text?

Answer: git log -S"text"

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Git internals: objects, refs and the index

12 minutes. Look inside a repository to see blobs, trees, commits, refs and the index, and understand why Git commands behave the way they do.

Day-to-day Git commands hide a simple data model. Looking at it once makes rebase, reset, detached HEAD and recovery far less mysterious.

What you will learn

- Blobs, trees, commits and tags
- Refs, HEAD and branches as pointers
- The index and what git add really does
- Using plumbing commands to inspect and recover

Objects

```
mkdir demo && cd demo && git init -q
echo "hello" > a.txt && git add a.txt && git commit -qm "first"
git cat-file -t HEAD           # commit
git cat-file -p HEAD           # tree, author, message
git cat-file -p 'HEAD^{tree}'  # the files in that snapshot
git cat-file -p "$(git rev-parse HEAD:a.txt)"   # the file contents
```

A **blob** holds file contents, a **tree** lists names and blobs or other trees, a **commit** points to a tree and parent commits and a **tag** names a commit. Each is stored under the hash of its content, so identical content is stored once and any change gives a new hash.

Refs and HEAD

```
cat .git/HEAD                  # ref: refs/heads/main
cat .git/refs/heads/main       # the commit hash
git branch feature             # creates a 41-byte file
git update-ref refs/heads/rescue 3f2a9c1
```

A **branch** is a file containing a commit hash. **HEAD** normally contains the name of the current branch, and committing moves that branch forward. In **detached HEAD** state, HEAD holds a hash directly, so new commits are not on any branch.

The index

```
git ls-files --stage
git diff --cached              # index versus last commit
git write-tree                 # turns the index into a tree object
```

The **index** is the staging area, a list of paths and blob hashes that will become the next commit's tree. `git add` writes a blob and updates the index. `git commit` writes a tree from the index and a commit object.

Recovery

```
git reflog                     # where HEAD and branches have been
git fsck --lost-found          # unreachable commits
git branch recovered a1b2c3d   # make them reachable again
```

Unreachable objects remain until garbage collection, usually for weeks, so lost commits are often recoverable.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Git internals: objects, refs and the index fits together

Part	What it does	What goes wrong
Objects	A blob holds file contents, a tree lists names and blobs or other trees, a commit points to a tree and parent commits and a tag names a commit.	Thinking branches are copies of files.
Refs and HEAD	A branch is a file containing a commit hash.	Committing in detached HEAD and moving away.
The index	The index is the staging area, a list of paths and blob hashes that will become the next commit's tree.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Recovery	Unreachable objects remain until garbage collection, usually for weeks, so lost commits are often recoverable.	Panicking after deleting a branch.

Flow: Objects to Refs and HEAD / Refs and HEAD to The index / The index to Recovery

Incident: The branch that seemed to vanish

Incident: *A deleted branch still held unmerged work, which was recovered.*

```
$ git branch -D feature/billing
Deleted branch feature/billing (was 9e41c7b).
reflog: 9e41c7b HEAD@{3}: commit: add invoice export
```

The developer deleted the branch before merging and panicked. Git had printed the hash of the tip. A new branch created at that commit restored the work immediately, and the team added a habit of reading the deletion message.

Common mistakes

- Thinking branches are copies of files
- Panicking after deleting a branch
- Committing in detached HEAD and moving away
- Running gc aggressively before checking the reflog

Check yourself

What is a branch?

Answer: A pointer to a commit

What does a commit point to?

Answer: A tree and its parent commits

What is the index?

Answer: The staging area for the next commit

How recover a lost commit?

Answer: Find it with reflog and branch from it

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. Release and hotfix workflows with Git

12 minutes. Cut releases from branches or tags, ship a hotfix without shipping unfinished work and keep long-lived branches in sync with cherry-picks.

The hardest release moment is the urgent fix. If you can recreate exactly what is in production from history, a small patch is routine. If you cannot, every fix is a risk.

What you will learn

- Tags and release branches
- Hotfix flow from the release tag
- Cherry-picking fixes across branches
- Automating versions and changelogs

Tag releases

```
git tag -s v2.8.0 -m "Release 2.8.0"
git push origin v2.8.0
git describe --tags --always
```

An **annotated, signed tag** records who released what and when. Build release artifacts from the tag, so production maps to an exact commit.

Hotfix

```
git switch -c hotfix/2.8.1 v2.8.0        # start from what is in production
# ...fix, test...
git commit -am "Fix invoice rounding (CVE-free, refs #812)"
git tag -s v2.8.1 -m "Hotfix 2.8.1"
git push origin hotfix/2.8.1 v2.8.1
git switch main && git merge --no-ff hotfix/2.8.1    # bring the fix back
```

Starting from the release tag means the hotfix contains only the fix. Afterwards merge it forward so main is not missing it.

Cherry-pick

```
git switch release/2.7
git cherry-pick -x 4b1c9e2
```

`-x` adds the original commit hash to the message, which helps trace where a fix came from. Backport the fix to each supported release branch, and resolve conflicts carefully, because the code may differ.

Automate

Use **Conventional Commits** so tools can derive the next version and a changelog from commit messages. Release from CI, not from laptops, and protect release branches and tags so they cannot be rewritten.

```
git log v2.8.0..v2.8.1 --oneline
git tag --contains 4b1c9e2
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Release and hotfix workflows with Git fits together

Part	What it does	What goes wrong
Tag releases	An annotated, signed tag records who released what and when.	Building fixes from main without a release tag.
Hotfix	Starting from the release tag means the hotfix contains only the fix.	Forgetting to merge hotfixes back.
Cherry-pick	-x adds the original commit hash to the message, which helps trace where a fix came from.	Cherry-picks without recording the source.
Automate	Use Conventional Commits so tools can derive the next version and a changelog from commit messages.	Unprotected release tags.

Flow: Tag releases to Hotfix / Hotfix to Cherry-pick / Cherry-pick to Automate

Incident: The hotfix that shipped half the backlog

Incident: *A fix was built from main and included unfinished features.*

```
built from: main (a91d3c0)
changes since v2.8.0: 47 commits
billing-v2 code path included, flag default on
```

There was no release tag to branch from, so the team built the fix from main, which carried many unreleased changes. A rollback was needed. They began tagging every release and branching hotfixes from the tag.

Common mistakes

- Building fixes from main without a release tag
- Forgetting to merge hotfixes back
- Cherry-picks without recording the source
- Unprotected release tags

Check yourself

Where start a hotfix from?

Answer: The release tag in production

What does cherry-pick -x add?

Answer: The original commit hash to the message

Why sign release tags?

Answer: They prove who released what

Why merge hotfixes forward?

Answer: So main also has the fix

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Git

Everyday commands for commits, branches, remotes, undoing mistakes, tags and recovery.

Set up and first commits

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global pull.ff only          # safe pull default
git init                                  # new repository here
git clone URL                             # copy a repository
git status                                # what is changed, staged, untracked
git add FILE                              # stage a file
git add -p                                # stage pieces of files
git commit -m "Fix retry loop in backup.sh"
git commit --amend --no-edit              # add staged changes to the last commit (unpushed)
```

Area	Meaning
Working tree	The files you edit
Staging area (index)	What goes into the next commit
Repository	The saved history in `.git`

Warning: `git restore FILE` *and* `git reset --hard` *throw away uncommitted edits for good. Run* `git status` *first, and stash if you are unsure.*

Look at history

```
git log --oneline --graph --decorate -15
git log -p -- FILE                        # changes to one file
git log -S"text"                          # commits that added or removed text
git log A..B                              # commits in B not in A
git show HASH                             # one commit in full
git diff                                  # working tree vs staging
git diff --staged                         # staging vs last commit
git diff main...feature                   # what the branch adds
git blame -L 40,60 FILE                   # who last changed each line
```

Branches and merging

```
git branch                                # list branches
git switch -c feature/retry               # create and switch
git switch main
git merge feature/retry                   # merge into current branch
git merge --no-ff feature/retry           # always create a merge commit
git merge --abort                         # give up a merge
git rebase origin/main                    # replay my commits on the latest main
git rebase -i HEAD~4                      # tidy the last four commits
git rebase --abort
git branch -d feature/retry               # delete a merged branch
git cherry-pick HASH                      # copy one commit here
```

Warning: *Never rebase or amend commits that others have already pulled. It creates new commits, and their copies no longer match.*

Remotes

```
git remote -v
git fetch origin                          # download, change nothing local
git pull --ff-only                        # fetch and fast-forward only
git pull --rebase                         # fetch and replay my commits
git push -u origin feature/retry          # first push, set upstream
git push --force-with-lease               # safer forced update (own branches only)
git branch -vv                            # ahead or behind
```

Warning: *Plain* `git push --force` *can erase a colleague's commits. Use* `--force-with-lease` *on your own branch, and block force pushes on shared branches.*

MARGA by EknathaLabs marga.eknathalabs.com

Undo

Goal	Command
Discard edits to a file	`git restore FILE`
Unstage a file	`git restore --staged FILE`
Fix last commit message	`git commit --amend`
Reverse a shared commit	`git revert HASH`
Undo commit, keep changes staged	`git reset --soft HEAD~1`
Match the remote exactly (loses local work)	`git reset --hard origin/main`
Park work	`git stash push -m "note"` then `git stash pop`
Preview deleting untracked files	`git clean -n`

Tags and releases

```
git tag -a v1.4.0 -m "Release 1.4.0"      # annotated tag
git push origin v1.4.0
git describe --tags                       # v1.4.0-12-g3f9c2ab
git log v1.3.0..v1.4.0 --oneline          # what is in the release
git tag --contains HASH                   # which releases hold this fix
```

Semantic versioning: `MAJOR.MINOR.PATCH`. Breaking change raises MAJOR, new compatible feature raises MINOR, bug fix raises PATCH.

Recover and investigate

```
git reflog                                # where HEAD has been
git branch rescue HASH                    # name a lost commit
git bisect start; git bisect bad; git bisect good v1.3.0
git bisect run ./test.sh                  # automatic search for the first bad commit
git fsck --lost-found                     # unreachable objects
git rm --cached FILE                      # stop tracking, keep the file
```

Big repositories

```
git clone --depth 1 URL                   # latest commit only
git clone --filter=blob:none URL          # fetch file contents on demand
git sparse-checkout set DIR1 DIR2         # check out only these folders
git worktree add ../hotfix hotfix/1.4     # second working folder
git submodule update --init --recursive
git count-objects -vH
```

Warning: *A secret that was committed is compromised, even after you delete it. Revoke it first, then clean history, because old clones keep a copy.*

Safe habits

- Run `git status` before every commit, reset or checkout.
- Keep commits small, with a message that says why.
- Never rewrite commits that others have pulled.
- Never commit secrets. If one is committed, revoke it first.
- Protect `main`: pull requests, review, required checks, no force push.

MARGA by EknathaLabs marga.eknathalabs.com